Project Document

Author: Adnan Ahmed

Social Media Engagement Analytics Using Python

1. Project Overview

Social media platforms generate massive amounts of engagement data such as likes, comments, shares, impressions, watch time, follower counts, and engagement rates. Analyzing this data helps identify content performance, user behavior, posting trends, and audience preferences.

This project uses Python and its data analytics libraries to clean, transform, analyze, visualize, and interpret a social media engagement dataset containing **5,000 records and 19 columns**.

The project covers data import, data cleaning, exploratory data analysis, data wrangling, statistical analysis, visualization, and business-oriented insights.

---

2. Problem Statement

Social media data contains valuable information about how users interact with different types of content. However, raw datasets may contain missing values, duplicate records, inconsistent data types, and unprocessed text fields.

The objective of this project is to clean and analyze the **social_media_engagement_5000.csv** dataset and identify meaningful patterns related to:

- Content performance
- User engagement
- Age groups
- Verified accounts
- Posting time
- Device usage
- Sentiment
- Country-wise engagement

The analysis uses **Pandas, NumPy, Matplotlib, Seaborn, and Plotly**.

---

3. Project Objectives

The main objectives of this project are:

1. Import and inspect the social media dataset.
2. Identify and handle missing values.
3. Detect and remove duplicate records.
4. Correct data types and standardize categorical values.
5. Extract useful features such as hashtag count.
6. Explore numerical and categorical variables.
7. Calculate descriptive statistics.
8. Analyze correlations between numerical variables.
9. Perform groupby-based analysis.
10. Combine summary DataFrames using merge/concat/join.
11. Create different types of visualizations.
12. Perform interactive visualization using Plotly.
13. Identify important engagement and behavioral patterns.
14. Present final findings supported by data analysis.

---

4. Dataset Description

Dataset Name: `social_media_engagement_5000.csv`

Number of Rows: 5,000
Number of Columns: 19

Column	Description
index	Record index
user_id	Unique user identifier
age	Age of the user
gender	Gender of the user
country	User's country
post_id	Unique post identifier
post_type	Type of social media post
post_category	Category of the post
likes	Number of likes
comments	Number of comments
shares	Number of shares
watch_time_sec	Watch time in seconds
impression_count	Number of impressions
posted_at	Date and time of posting
follower_count	Number of followers
is_verified	Verification status

Column	Description
device_type	Device used to access content
sentiment	Sentiment of the post
hashtags	Hashtags associated with the post
engagement_rate	Engagement rate

---

5. Technologies Used

The following tools and libraries were used:

- **Python**
- **Pandas** – Data manipulation and analysis
- **NumPy** – Numerical operations
- **Matplotlib** – Data visualization
- **Seaborn** – Statistical visualization
- **Plotly** – Interactive visualization
- **Google Colab / Jupyter Notebook**

---

6. Task 1 – Data Import & Setup

6.1 Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
```

Explanation

The required Python libraries are imported before starting the analysis.

- Pandas is used for DataFrame operations.
- NumPy is used for numerical operations.
- Matplotlib and Seaborn are used for visualization.
- Plotly is used to create interactive charts.

6.2 Import the CSV Dataset

```
df = pd.read_csv("social_media_engagement_5000.csv")
```

Explanation

The CSV file is loaded into a Pandas DataFrame named `df`.

6.3 Check Dataset Shape

```
df.shape
```

Output

```
(5000, 19)
```

Explanation

The dataset contains **5,000 rows and 19 columns**.

6.4 View First Records

```
df.head()
```

This displays the first five records of the dataset.

6.5 View Last Records

```
df.tail()
```

This displays the last five records.

6.6 View Column Names

```
df.columns
```

This displays all column names available in the dataset.

6.7 Check Data Types

```
df.dtypes
```

This helps identify whether columns are stored as integer, float, object, boolean, or another data type.

6.8 Dataset Information

```
df.info()
```

This provides information about:

- Number of records
- Column names
- Data types
- Non-null values
- Memory usage

---

7. Task 2 – Data Cleaning

7.1 Detect Missing Values

```
df.isnull().sum()
```

The initial missing values were:

Column	Missing Values
age	150
gender	150
likes	150
comments	150
shares	150
sentiment	150

7.2 Handle Missing Numerical Values

Numerical columns were filled using their median values.

```
for col in ["age", "likes", "comments", "shares"]:
    df[col] = df[col].fillna(df[col].median())
```

Explanation

Median was selected because it is less affected by extreme values compared with the mean.

7.3 Handle Missing Categorical Values

```
for col in ["gender", "sentiment"]:
    df[col] = df[col].fillna(df[col].mode()[0])
```

Explanation

The mode represents the most frequently occurring category, making it suitable for filling missing categorical values.

7.4 Check Duplicate Records

```
df.duplicated().sum()
```

Output

```
0
```

Explanation

No duplicate records were found in the dataset.

7.5 Convert Date Column

```
df["posted_at"] = pd.to_datetime(df["posted_at"])
```

Explanation

The `posted_at` column was converted from text/object format to datetime format so that date and time-based analysis could be performed.

7.6 Convert Numerical Columns

```
for col in ["age", "likes", "comments", "shares"]:
    df[col] = df[col].astype(int)
```

This converts the selected numerical columns into integer format.

7.7 Standardize Sentiment

```
df["sentiment"] = df["sentiment"].str.strip().str.lower()
```

Explanation

This removes unnecessary spaces and converts sentiment values to lowercase to maintain consistency.

7.8 Extract Hashtag Count

```
df["hashtag_count"] = df["hashtags"].fillna("").apply(
    lambda x: len(x.split())
)
```

Explanation

A new column called `hashtag_count` was created to represent the number of hashtags associated with each post.

---

8. Task 3 – Data Exploration Using Pandas

8.1 Summary Statistics

```
df.describe()
```

This provides statistical information such as:

- Count
- Mean
- Standard deviation
- Minimum
- 25th percentile
- Median
- 75th percentile
- Maximum

8.2 Categorical Analysis

The following functions were used:

```
df["gender"].unique()
df["gender"].nunique()
df["gender"].value_counts()
```

Similar analysis was performed for:

- Country
- Post type
- Post category
- Device type
- Sentiment
- Verification status

8.3 Correlation Matrix

```
df.select_dtypes(include="number").corr()
```

Explanation

The correlation matrix helps identify linear relationships between numerical variables.

Correlation values range from **-1 to +1**.

A correlation close to +1 indicates a strong positive relationship, while a value close to -1 indicates a strong negative relationship.

Correlation does not by itself prove causation.

---

9. Groupby Analysis

9.1 Engagement by Post Type

```
df.groupby("post_type")["engagement_rate"].mean().sort_values(
    ascending=False
)
```

Result

Post Type	Average Engagement Rate
Video	1.122
Text	1.065
Image	0.896
Reel	0.783

Insight

Video posts recorded the highest average engagement rate.

9.2 Engagement by Content Category

```
df.groupby("post_category")["engagement_rate"].mean().sort_values(
    ascending=False
)
```

Key Result

- Food – approximately **1.359**
- Tech – approximately **1.160**
- Lifestyle – approximately **1.091**

Insight

Food content recorded the highest average engagement rate among the categories analyzed.

9.3 Engagement by Country

```
df.groupby("country")["engagement_rate"].mean().sort_values(
    ascending=False
)
```

Key Results

Country	Average Engagement Rate
Brazil	1.541
Australia	1.324
France	1.146
UAE	1.112

Insight

Brazil recorded the highest average engagement rate in the dataset.

---

10. Task 4 – Data Wrangling

10.1 Create Age Groups

```
bins = [0, 18, 25, 35, 45, 55, 65, 100]

labels = [
    "<18",
    "18-25",
    "26-35",
    "36-45",
    "46-55",
    "56-65",
    "65+"
]

df["age_group"] = pd.cut(
    df["age"],
    bins=bins,
    labels=labels
)
```

The new `age_group` feature allows engagement to be compared across different age ranges.

10.2 Age Group Engagement

```
df.groupby(
    "age_group",
    observed=True
)["engagement_rate"].mean()
```

Insight

The **46–55** and **18–25** age groups showed relatively higher average engagement, while the **56–65** group showed the lowest average engagement among the observed groups.

---

11. Verified Account Analysis

```
df.groupby("is_verified")["engagement_rate"].mean()
```

Results

Account Type	Average Engagement Rate
Verified	1.054
Unverified	0.955

Insight

Verified accounts recorded a slightly higher average engagement rate, with a difference of approximately **0.099 engagement-rate points**.

---

12. Posting Time Analysis

A new posting-hour field was created:

```
df["post_hour"] = df["posted_at"].dt.hour
```

Average impressions by hour:

```
df.groupby("post_hour")["impression_count"].mean().sort_values(
    ascending=False
)
```

Result

The highest observed average impression count occurred around **12:00 AM (midnight)**, at approximately **50,014 impressions**.

Insight

Posts made around midnight recorded the highest average impressions among the observed posting hours.

---

13. Device Type and Watch Time

```
df.groupby("device_type")["watch_time_sec"].mean().sort_values(
    ascending=False
)
```

Results

Device	Average Watch Time
Mobile	4,087.83 sec
Tablet	3,979.74 sec
Desktop	3,974.79 sec

Insight

Mobile devices recorded the highest average watch time. This indicates that users on mobile devices spent slightly more time watching content than users on tablets and desktops.

---

14. Sentiment Analysis

14.1 Engagement by Sentiment

```
df.groupby("sentiment")["engagement_rate"].mean().sort_values(
    ascending=False
)
```

Results

Sentiment	Average Engagement Rate
Negative	1.038
Neutral	0.991

Sentiment Average Engagement Rate
Positive 0.919

Insight

Negative sentiment posts recorded the highest average engagement rate in this dataset.

---

15. Negative vs Neutral Sentiment

```
df[
    df["sentiment"].isin(["negative", "neutral"])
].groupby("sentiment")[
    ["likes", "comments", "shares",
     "impression_count", "engagement_rate"]
].mean()
```

Results

Metric	Negative	Neutral
Likes	10,208	9,923
Comments	1,516	1,528
Engagement Rate	1.038	0.991

Average shares and impression counts were similar for both sentiment types.

Insight

Negative sentiment posts recorded slightly higher average likes and engagement rates, while neutral posts recorded slightly more average comments.

---

16. Task 5 – Statistical Analysis

The following columns were analyzed:

- Likes
- Comments
- Shares
- Watch time
- Engagement rate

- Follower count

```
columns = [
    "likes",
    "comments",
    "shares",
    "watch_time_sec",
    "engagement_rate",
    "follower_count"
]

stats = pd.DataFrame({
    "Mean": df[columns].mean(),
    "Median": df[columns].median(),
    "Mode": df[columns].mode().iloc[0],
    "Standard Deviation": df[columns].std(),
    "Variance": df[columns].var(),
    "25th Percentile": df[columns].quantile(0.25),
    "50th Percentile": df[columns].quantile(0.50),
    "75th Percentile": df[columns].quantile(0.75),
    "90th Percentile": df[columns].quantile(0.90),
    "Skewness": df[columns].skew(),
    "Kurtosis": df[columns].kurt()
})

stats.round(2)
```

Known descriptive results

Metric	Mean	Median
Likes	10,107.04	10,105.50
Comments	1,502.20	1,497
Shares	1,002.63	1,012
Engagement Rate	0.964	0.2539
Age	38.454	38

The engagement-rate variable showed a substantial difference between mean and median, with an observed maximum of approximately **191.50**, indicating the presence of extreme values.

---

17. DataFrame Combining

The `merge()` function was used to combine summary DataFrames.

```
likes_summary = df.groupby(
    "post_type"
)["likes"].mean().reset_index(name="avg_likes")
```

```
engagement_summary = df.groupby(
    "post_type"
)["engagement_rate"].mean().reset_index(
    name="avg_engagement"
)

final_summary = pd.merge(
    likes_summary,
    engagement_summary,
    on="post_type"
)

final_summary
```

Explanation

Two separate summaries were created and combined using the common `post_type` column.

`concat()` and `join()` can also be used for combining compatible DataFrames.

---

18. Task 6 – Data Visualization

Matplotlib Visualizations

The project includes the following Matplotlib charts:

1. Scatter plot – Likes vs Impressions
2. Line chart – Daily Engagement Trend
3. Bar chart – Posts by Category
4. Pie chart – Gender Distribution
5. Histogram – Age Distribution
6. Box plot – Engagement Rate
7. Bar chart – Average Engagement by Post Type
8. Bar chart – Average Watch Time by Device

Seaborn Visualizations

The following Seaborn visualizations were created:

1. Count plot – Post Type
2. Bar plot – Average Likes by Category
3. Violin plot – Followers vs Sentiment

4. Pair plot – Numerical Features
5. Heatmap – Correlation Matrix
6. Swarm plot – Engagement vs Device

Plotly Interactive Visualization

An interactive daily engagement chart was created using Plotly:

```
daily_engagement = (
    df.groupby(df["posted_at"].dt.date)["engagement_rate"]
    .mean()
    .reset_index()
)

daily_engagement.columns = [
    "date",
    "avg_engagement"
]

fig = px.line(
    daily_engagement,
    x="date",
    y="avg_engagement",
    title="Interactive Daily Engagement Trend",
    markers=True
)

fig.update_layout(
    xaxis_title="Date",
    yaxis_title="Average Engagement Rate"
)

fig.show()
```

This allows users to interact with the chart and examine engagement trends over time.

---

19. Final Insights

Content Performance

Post Type:
Video posts recorded the highest average engagement rate at approximately **1.122**, followed by text, image, and reel posts.

Content Category:
The **food category** recorded the highest average engagement rate at approximately **1.359**.

Country:
Brazil recorded the highest average engagement rate at approximately **1.541**, followed by Australia, France, and UAE.

User Trends

Age:
The **46–55** and **18–25** age groups showed relatively higher average engagement, while the **56–65** group showed the lowest average engagement among the observed groups.

Verified Accounts:
Verified accounts recorded an average engagement rate of approximately **1.054**, compared with **0.955** for unverified accounts.

Behavioral Insights

Best Time for Impressions:
The highest observed average impression count occurred around **12:00 AM**, with approximately **50,014 impressions**.

Device Impact:
Mobile users recorded the highest average watch time at approximately **4,087.83 seconds**, followed by tablets and desktops.

Sentiment Analysis

Best Performing Sentiment:
Negative sentiment posts recorded the highest average engagement rate at approximately **1.038**, followed by neutral and positive posts.

Negative vs Neutral:
Negative posts recorded slightly higher average likes and engagement rates, while neutral posts recorded slightly more average comments. Their average shares and impression counts were similar.

---

20. Overall Conclusion

This project demonstrates the use of Python for end-to-end social media data analysis. The dataset was cleaned and transformed before performing exploratory analysis, statistical analysis, data wrangling, and visualization.

The analysis identified differences in engagement across **post types, content categories, countries, age groups, verification status, posting times, devices, and sentiment**.

The results show that video posts, food-related content, and certain audience segments recorded relatively higher engagement within this dataset. Mobile devices showed the highest average watch time, while midnight recorded the highest observed average impressions. Negative sentiment posts also showed higher average engagement compared with neutral and positive posts in this dataset.

Overall, the project demonstrates how raw social media data can be transformed into structured insights using Python data analytics techniques